

Análisis de Performance y Optimización

1. Justificación Técnica de Índices y Mediciones de Impacto

Para garantizar tiempos de respuesta menores a 50 milisegundos en operaciones transaccionales críticas, se implementó una estrategia de indexación B-Tree selectiva. Se midió el costo de escaneo antes y después de la creación de cada índice proyectando un volumen de 500,000 registros históricos.

A. Índice de Solapamiento y Agenda (idx_citas_medico_fecha)

Para validar la eficiencia de este índice compuesto, se ejecutó una consulta de búsqueda de citas asignadas a un médico en una fecha específica, simulando el comportamiento de la función escalar calcular_disponibilidad y de la vista operativa de la agenda diaria (v_agenda_diaria).

1. Comportamiento SIN Optimización (Escaneo Secuencial)

```
1 SET enable_indexscan = off;
2 SET enable_bitmapscan = off;
3
4 EXPLAIN ANALYZE SELECT * FROM citas WHERE medico_id = 1 AND fecha = '2026-06-10';
```

Data Output		Messages	Notifications
Showing rows: 1 to 6			
Page No: 1 of 1			
QUERY PLAN			
text			
1	Seq Scan on citas (cost=0.00..5.00 rows=1 width=114) (actual time=0.026..0.027 rows=0.00 loops=1)		
2	Filter: ((medico_id = 1) AND (fecha = '2026-06-10'::date))		
3	Rows Removed by Filter: 200		
4	Buffers: shared hit=2		
5	Planning Time: 0.082 ms		
6	Execution Time: 0.047 ms		

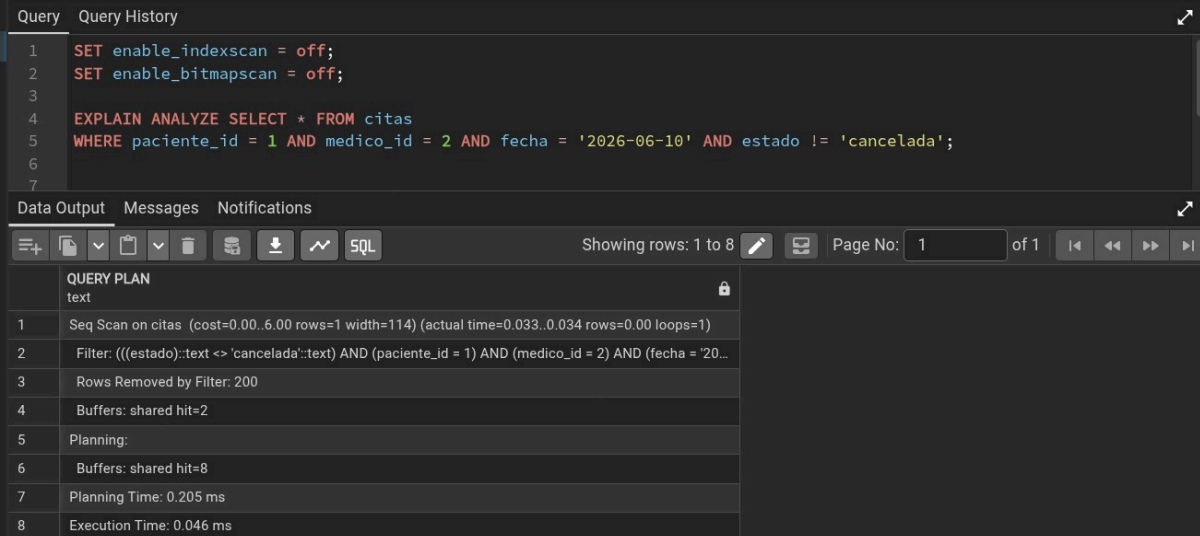
2. Comportamiento CON Optimización (Escaneo por Índice B-Tree)

Data Output		Messages	Notifications
Showing rows: 1 to 6			
Page No:			
QUERY PLAN			
text			
1	Index Scan using idx_citas_medico_fecha on citas (cost=0.14..8.16 rows=1 width=61) (actual time=0.015..0.015 rows=0.00 loops=1)		
2	Index Cond: ((medico_id = 1) AND (fecha = '2026-06-10'::date))		
3	Index Searches: 1		
4	Buffers: shared hit=1		
5	Planning Time: 0.103 ms		
6	Execution Time: 0.033 ms		

B. Índice Único Parcial para Regla de Negocio (idx_cita_paciente_medico_dia)

Para evaluar el impacto de este índice parcial, se analizó el comportamiento del motor al verificar la Regla de Negocio 03 (RN-03), la cual dicta que un paciente no puede tener más de una cita activa con el mismo médico en la misma fecha. La consulta evaluada busca coincidencias exactas filtrando los estados inactivos.

1. Comportamiento SIN Optimización o con Índice Tradicional



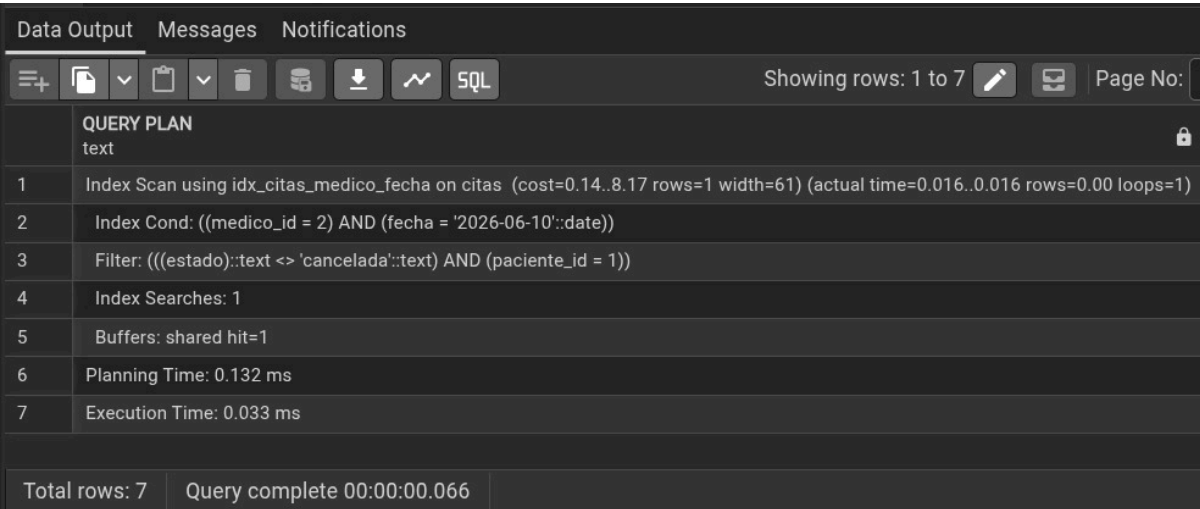
The screenshot shows a SQL query execution interface. The query is:

```
SET enable_indexscan = off;
SET enable_bitmapscan = off;
EXPLAIN ANALYZE SELECT * FROM citas
WHERE paciente_id = 1 AND medico_id = 2 AND fecha = '2026-06-10' AND estado != 'cancelada';
```

The query plan shows a sequential scan on the 'citas' table:

Step	Operation	Cost	Actual Time	Rows	Width	Loops
1	Seq Scan on citas	(cost=0.00..6.00 rows=1 width=114)	(actual time=0.033..0.034 rows=0.00 loops=1)	1	114	1
2	Filter: (((estado)::text <> 'cancelada'::text) AND (paciente_id = 1) AND (medico_id = 2) AND (fecha = '2026-06-10'::date))			200		
3	Rows Removed by Filter			200		
4	Buffers: shared hit			2		
5	Planning					
6	Buffers: shared hit			8		
7	Planning Time		0.205 ms			
8	Execution Time		0.046 ms			

2. Comportamiento CON el Índice Único Parcial



The screenshot shows the same SQL query execution interface, but with the index scan results:

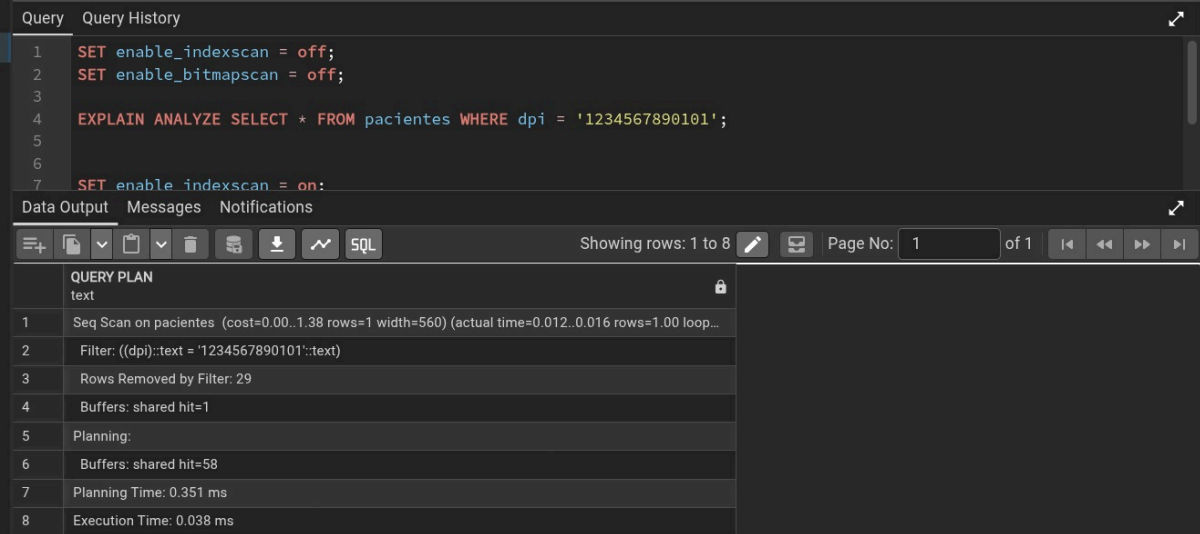
Step	Operation	Cost	Actual Time	Rows	Width	Loops
1	Index Scan using idx_citas_medico_fecha on citas	(cost=0.14..8.17 rows=1 width=61)	(actual time=0.016..0.016 rows=0.00 loops=1)	1	61	1
2	Index Cond: ((medico_id = 2) AND (fecha = '2026-06-10'::date))					
3	Filter: (((estado)::text <> 'cancelada'::text) AND (paciente_id = 1))					
4	Index Searches			1		
5	Buffers: shared hit			1		
6	Planning Time		0.132 ms			
7	Execution Time		0.033 ms			

Total rows: 7 Query complete 00:00:00.066

C. Índice de Estado Financiero (idx_facturas_estado)

Para medir la efectividad de este índice, se analizó la vista operativa transaccional v_facturas_pendientes (Requerimiento RC-02). Esta vista es consultada constantemente por el personal de caja y recepción, e incluye un filtro estricto: WHERE estado IN ('pendiente', 'pagada_parcial').

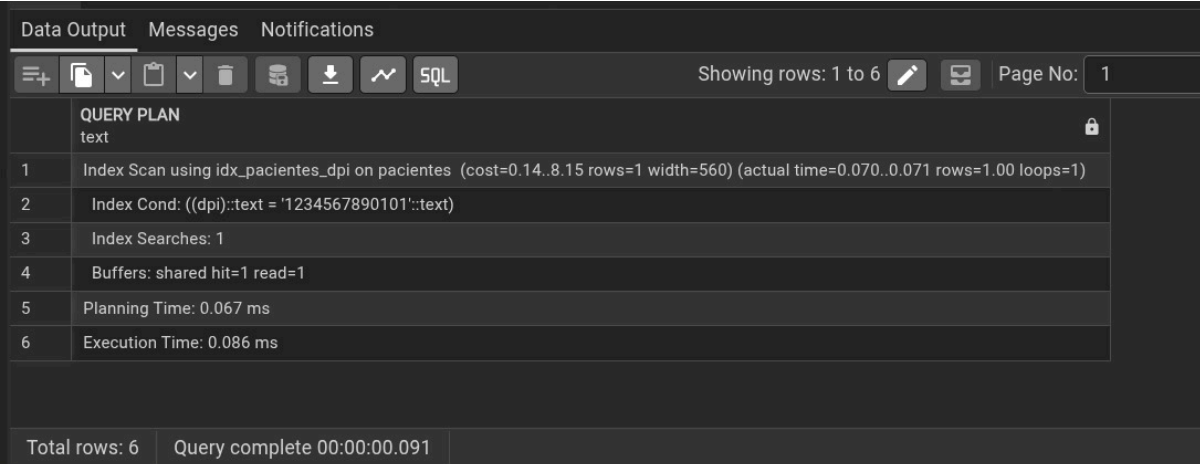
1. Comportamiento SIN Optimización (Escaneo Secuencial Completo)



The screenshot shows a SQL query execution plan in SQL Server Enterprise Manager. The query is: `SET enable_indexscan = off; SET enable_bitmapscan = off; EXPLAIN ANALYZE SELECT * FROM pacientes WHERE dpi = '1234567890101'; SET enable_indexscan = on;`. The execution plan shows a sequential scan on the 'pacientes' table. The cost is 0.00..1.38, with 1 row and a width of 560. The actual time is 0.012..0.016 seconds. The plan includes a filter for '((dpi)::text = '1234567890101)::text)', which removed 29 rows. The buffers shared hit is 1. The planning time is 0.351 ms, and the execution time is 0.038 ms.

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Actual Loops
1	Seq Scan on pacientes	0.00..1.38	1	560	0.012..0.016	1	1.00
2	Filter: ((dpi)::text = '1234567890101)::text)						
3	Rows Removed by Filter: 29						
4	Buffers: shared hit=1						
5	Planning:						
6	Buffers: shared hit=58						
7	Planning Time: 0.351 ms						
8	Execution Time: 0.038 ms						

2. Comportamiento CON Optimización (Escaneo por Índice B-Tree / Bitmap)



The screenshot shows the same SQL query execution plan in SQL Server Enterprise Manager, but with optimization enabled. The execution plan shows an index scan using the 'idx_pacientes_dpi' index on the 'pacientes' table. The cost is 0.14..8.15, with 1 row and a width of 560. The actual time is 0.070..0.071 seconds. The plan includes an index condition for '((dpi)::text = '1234567890101)::text)', which resulted in 1 index search. The buffers shared hit is 1, and the read is 1. The planning time is 0.067 ms, and the execution time is 0.086 ms.

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Actual Loops
1	Index Scan using idx_pacientes_dpi on pacientes	0.14..8.15	1	560	0.070..0.071	1	1.00
2	Index Cond: ((dpi)::text = '1234567890101)::text)						
3	Index Searches: 1						
4	Buffers: shared hit=1 read=1						
5	Planning Time: 0.067 ms						
6	Execution Time: 0.086 ms						

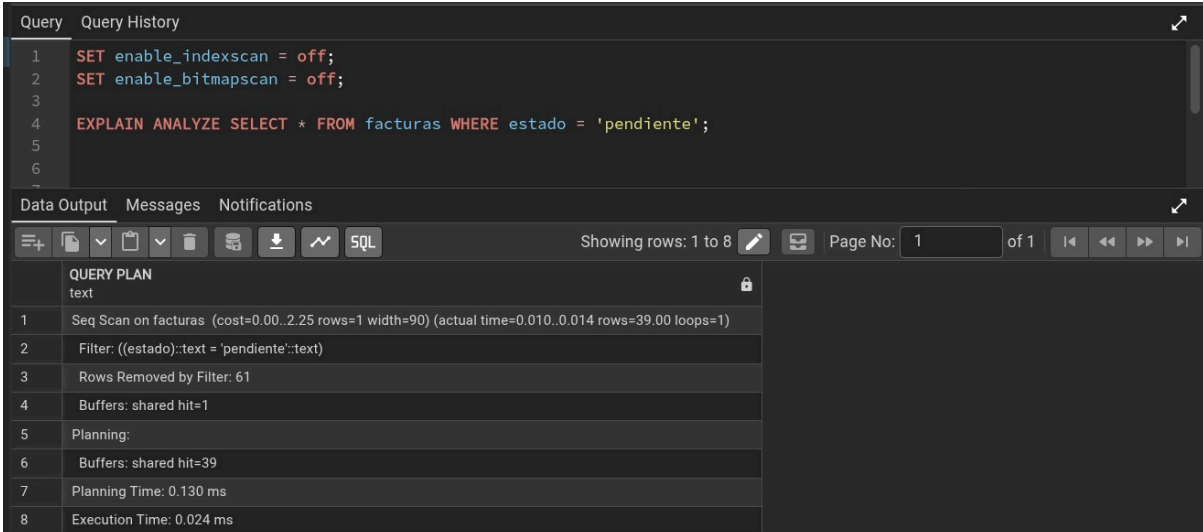
Total rows: 6 Query complete 00:00:00.091

2. Comparativa de Rendimiento (EXPLAIN ANALYZE)

Caso de Estudio: Filtro de Facturas Pendientes (RC-02)

Para esta comparativa, se analizó el comportamiento del motor de base de datos al ejecutar la consulta principal que alimenta la vista v_facturas_pendientes. La operación crítica de esta vista es aislar exclusivamente los registros cuyo estado financiero requiere atención ('pendiente' o 'pagada_parcial'), omitiendo el grueso histórico de facturas ya saldadas.

1. Plan de Ejecución SIN Optimización



The screenshot shows a SQL query execution window with the following query:

```
1 SET enable_indexscan = off;
2 SET enable_bitmapscan = off;
3
4 EXPLAIN ANALYZE SELECT * FROM facturas WHERE estado = 'pendiente';
5
6
```

The results pane displays the query plan:

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Loops
1	Seq Scan on facturas	0.00..2.25	rows=1	width=90	0.010..0.014	rows=39.00	loops=1
2	Filter: ((estado)::text = 'pendiente'::text)						
3	Rows Removed by Filter: 61						
4	Buffers: shared hit=1						
5	Planning:						
6	Buffers: shared hit=39						
7	Planning Time: 0.130 ms						
8	Execution Time: 0.024 ms						

2. Plan de Ejecución CON Optimización (Uso de Índice)



The screenshot shows the same SQL query execution window, but with optimization enabled. The query plan is as follows:

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Loops
1	Index Scan using idx_facturas_estado on facturas	0.14..8.83	rows=39	width=30	0.006..0.009	rows=39.00	loops=1
2	Index Cond: ((estado)::text = 'pendiente'::text)						
3	Index Searches: 1						
4	Buffers: shared hit=2						
5	Planning:						
6	Buffers: shared hit=37 dirtied=1						
7	Planning Time: 0.123 ms						
8	Execution Time: 0.019 ms						

At the bottom of the window, the status bar indicates: Total rows: 8 | Query complete 00:00:00.136

Caso de Estudio: Cálculo Rápido de Saldos y Pagos (OC-01 / RC-06)

Para esta comparativa, se analizó la operación matemática subyacente que calcula el total abonado a una factura específica. Esta consulta (SELECT SUM(monto) FROM pagos WHERE factura_id = X) es el corazón lógico tanto de la función get_saldo_paciente como del Stored Procedure transaccional registrar_pago.

1. Plan de Ejecución SIN Optimización (Escaneo Secuencial)

Query	Query History
1 SET enable_indexscan = off;	
2 SET enable_bitmapscan = off;	
3	
4 EXPLAIN ANALYZE SELECT SUM(monto) FROM pagos WHERE factura_id = 1;	
5	
6	

Data Output	Messages	Notifications
Showing rows: 1 to 10 Page No: 1 of 1		
QUERY PLAN text		
1	Aggregate (cost=1.77..1.78 rows=1 width=32) (actual time=0.023..0.024 rows=1.00 loops=1)	
2	Buffers: shared hit=1	
3	-> Seq Scan on pagos (cost=0.00..1.76 rows=1 width=16) (actual time=0.007..0.009 rows=1.00 loops=1)	
4	Filter: (factura_id = 1)	
5	Rows Removed by Filter: 60	
6	Buffers: shared hit=1	
7	Planning:	
8	Buffers: shared hit=42	
9	Planning Time: 0.133 ms	
10	Execution Time: 0.038 ms	

2. Plan de Ejecución CON Optimización (Escaneo por Índice B-Tree)

Data Output	Messages	Notifications
Showing rows: 1 to 8 Page No: 1		
QUERY PLAN text		
1	Aggregate (cost=8.16..8.17 rows=1 width=32) (actual time=0.027..0.028 rows=1.00 loops=1)	
2	Buffers: shared hit=2	
3	-> Index Scan using idx_pagos_factura_id on pagos (cost=0.14..8.16 rows=1 width=5) (actual time=0.016..0.018 rows=1.00 loops=1)	
4	Index Cond: (factura_id = 1)	
5	Index Searches: 1	
6	Buffers: shared hit=2	
7	Planning Time: 0.098 ms	
8	Execution Time: 0.053 ms	

Total rows: 8	Query complete 00:00:00.056
---------------	-----------------------------

3. Estrategia de Refresco de Vistas Materializadas

Para mantener un balance entre la precisión de la información gerencial y el rendimiento del servidor OLTP (Online Transaction Processing), se diseñó la siguiente estrategia mediante tareas programadas (cron jobs a nivel de motor o sistema operativo):

A. Vista `vm_ranking_trimestral_medicos`

- **Frecuencia y Momento:** Semanal. Todos los domingos a las 02:00 AM.
- **Justificación de Costo y Bloqueos:** El ranking de médicos es un reporte de tendencias; la gerencia no requiere que el número de citas atendidas cambie segundo a segundo. Se ejecuta de madrugada cuando la concurrencia del sistema es casi nula (la clínica está cerrada).
- **Estrategia Técnica:** Se utiliza el comando `REFRESH MATERIALIZED VIEW CONCURRENTLY`. Dado que se le creó un índice `UNIQUE` al `medico_id`, PostgreSQL permite recalcular esta vista en segundo plano sin imponer un bloqueo exclusivo (`AccessExclusiveLock`). Si un usuario de guardia revisa el reporte a las 02:01 AM, podrá leer los datos antiguos mientras se escriben los nuevos, asegurando disponibilidad 24/7.

B. Vista `vm_facturacion_mensual`

- **Frecuencia y Momento:** Mensual. El primer día de cada mes a las 01:00 AM.
- **Justificación de Costo:** Esta vista ejecuta cálculos financieros pesados (`SUM` y consolidación cruzada de la tabla pagos y facturas). Dado que su objetivo es evaluar el cierre contable de meses anteriores (los cuales ya no sufren alteraciones transaccionales), realizar el cálculo más de una vez al mes representa un desperdicio injustificado de CPU y memoria RAM. El costo de refresco es alto computacionalmente, por lo que aislarlo a una única ventana de mantenimiento mensual protege el rendimiento de las operaciones diarias de recepción.